

FPGA(ZCU104) Based Energy Efficient Accelerator for MobileNet-V1

Yanamala Rama Muni Reddy

Student Member, IEEE

Dept of ECE

NIT Warangal

Warangal, India

yanamalamunireddy@gmail.com

Pullakandam Muralidhar

Senior Member, IEEE

Dept of ECE

NIT Warangal

Warangal, India

pmurali@nitw.ac.in

G.N.V.Satya Narayana

Dept of ECE

GMRIT

Rajam, India

venkatgannamani455@gmail.com

Dumpala Jagan

Dept of ECE

GMRIT

Rajam, India

dumpalajagan540@gmail.com

Abstract— Convolutional neural networks (CNNs) are commonly used in modern AI systems. These models feature millions of layer connections, which are memory- and computationally-intensive. Using these models on an embedded mobile application requires a lot of power and bandwidth to retrieve off-chip DRAM data. Reducing data transmission between on-chip and off-chip DRAM is key to high throughput and energy efficiency. The proposed FPGA-based hardware accelerator achieves these goals by employing a patch-wise design that uses less on-chip memory. The proposed architecture involves parallel computations on the patches stored in the on-chip memory. The proposed architecture is verified with a synthetic aperture radar (SAR) dataset, crucial in military surveillance, disaster management, and marine vigilance applications. The proposed accelerator prevented the need for a cloud-based server and was implemented on the Zynq ultra-scale ZCU104 FPGA board. The proposed architecture consumed 81% less power than GPU and 44% less than state-of-the-art works, underscoring its eco-friendliness. Furthermore, the proposed architecture achieved 24 GOP/s.

Keywords— deep learning, mobile computer vision, Mobilenet-v1, hardware accelerator

I. INTRODUCTION

In deep learning, substantial progress has been made across various domains, with Convolutional Neural Networks (CNNs) emerging as a critical innovation. Google's 2017 release of MobileNet [1] was a significant milestone, addressing a common challenge in traditional CNN models – their large size while maintaining a balance between accuracy and resource efficiency [2].

SAR (Synthetic Aperture Radar) datasets present complex data that demands substantial computational power for efficient processing. Hardware accelerators like mobile devices and compact hardware equipped with specialized processors such as GPUs or TPUs have proven indispensable to achieve real-time capabilities. Their portability, energy efficiency, and cost-effectiveness make them well-suited for remote or rugged environments lacking conventional computing infrastructure. Simultaneously, FPGA-based implementations of MobileNet have emerged, primarily focusing on enhancing classification speed [4-9]. However, many of these initiatives need to pay more attention to hardware structure optimization, resulting in inefficient

resource utilization. The authors [4-10] introduce a meticulously designed high-performance FPGA hardware architecture and acceleration unit by increasing the usage of resources. The proposed architecture uses a patchwise design and consumes less power. This architecture seamlessly integrates with existing MobileNet models while delivering superior performance. A rigorous analysis of the results validates the proposed solution's effectiveness and efficiency. By combining CNNs like MobileNet with hardware accelerators tailored for SAR data processing, new possibilities for real-time, resource-efficient applications across diverse domains are unlocked.

II. BACKGROUND

MobileNet is designed to be a lightweight and efficient neural network for tasks like image classification on resource-constrained devices. Its primary features include maintaining the Integrity of the Specifications.

Table 1. MobileNet-V1 Body Architecture

Type/Stride	Input Shape	Filter Shape	Output Shape
Conv / s2	225×225×3	3×3×3×32	112×112×32
Conv dw / s1	114×114×32	3×3×32 dw	112×112×32
Conv / s1	112×112×32	1×1×32×64	112×112×64
Conv dw / s2	113×113×64	3×3×64 dw	56×56×64
Conv / s1	56×56×64	1×1×64×128	56×56×128
Conv dw / s1	58×58×128	3×3×128 dw	56×56×128
Conv / s1	56×56×128	1×1×128×128	56×56×128
Conv dw / s2	57×57×128	3×3×128 dw	28×28×128
Conv / s1	28×28×128	1×1×128×256	28×28×256
Conv dw / s1	30×30×256	3×3×256 dw	28×28×256
Conv / s1	28×28×256	1×1×256×256	28×28×256
Conv dw / s2	29×29×256	3×3×256 dw	14×14×256
Conv / s1	14×14×256	1×1×256×512	14×14×512
Conv dw / s1	16×16×512	3×3×512 dw	14×14×512
Conv / s1	14×14×512	1×1×512×512	14×14×512
Conv dw / s2	15×15×512	3×3×512 dw	7×7×512
Conv / s1	7×7×512	1×1×512×1024	7×7×1024
Conv dw / s1	9×9×1024	3×3×1024 dw	7×7×1024
Conv / s1	7×7×1024	1×1×1024×1024	7×7×1024
Avg Pool / s1	7×7×1024	Pool 7×7	1×1×1024
FC / s1	1×1×1024	1024×8	1×1×8
Softmax / s1	1×1×8	Classifier	1×1×1

MobileNet is a convolutional neural network (CNN) architecture that uses depthwise separable convolutions to build a lightweight deep neural network. The architecture is designed for mobile and embedded vision applications.

Mobilenet is efficient and portable and is used in real-world applications. It requires very little computation power to run or apply transfer learning. The MobileNet model has 13 million parameters. The network consists of 28 convolution layers and one fully connected layer followed by a softmax layer, as shown in Fig. 1. Table I mentions The input, output, and filter shapes.

The convolution layers are arranged such that depthwise convolution is followed by a pointwise convolution layer, which internally contains Batch Normalization and ReLu activation functions, as shown in Fig. 2(b). The same is compared with standard convolution shown in Fig. 2(a).



Fig.1 Mobilenet-V1 Architecture Block Diagram

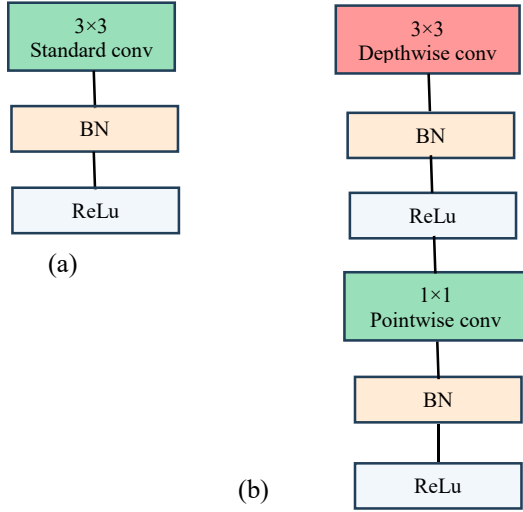


Fig.2 Standard and Depthwise Convolution Block Diagram

A. Standard Convolution

Standard convolution is a fundamental operation in deep learning, prominently employed in convolutional neural networks (CNNs). This process systematically applies a filter or kernel across an input volume, calculating weighted sums within localized receptive fields. Crucially, this computation occurs independently for each input channel, with distinct learnable weight sets for each output channel, as shown in Fig. 3.

Its significance lies in its ability to uncover spatial hierarchies and intricate data features, making it essential for various image and signal processing tasks. CNNs leverage standard convolution to learn diverse patterns, from bare edges and textures to complex objects and structures. The expression for Standard convolution is given by Eq. (1).

$$\text{Output}[i, j, K] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} \sum_{l=0}^{K-1} \text{Input}[i+m, j+n, K+l] \times \text{Kernel}[m, n, l] \quad (1)$$

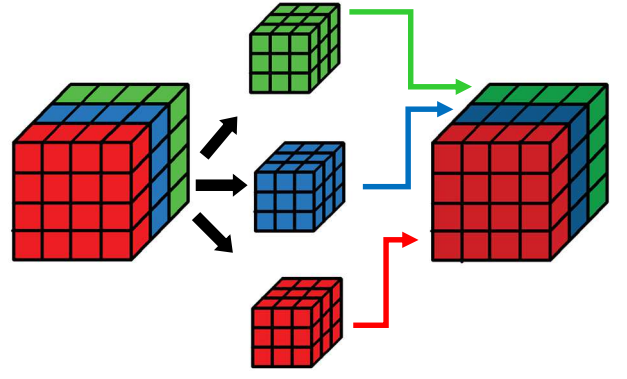


Fig.3 Standard convolution

However, this operation introduces many learnable parameters, resulting in computational and memory demands. Various variants like depthwise and separable convolution have been developed to reduce parameter counts and computational complexity to address this. Despite its computational intensity, standard convolution remains indispensable in applications where maximizing model performance takes precedence over computational efficiency. This includes fields requiring high precision, such as image recognition, natural language processing, and advanced computer vision applications.

B. Depthwise separable convolution

Depthwise separable convolution, a critical component of modern neural network architectures, represents a computationally efficient and parameter-sparing convolutional operation. This method decomposes the standard convolution into two distinct steps, shown in Figs. 4 and 5.

Depthwise convolution applies a separate convolution to each input channel, followed by pointwise convolution, which combines the results through 1x1 convolutions. This separation significantly reduces computation, making it well-suited for resource-constrained environments while maintaining high model performance. It has become integral in various applications, from mobile and edge devices to real-time computer vision tasks, where optimizing accuracy and computational efficiency is paramount.

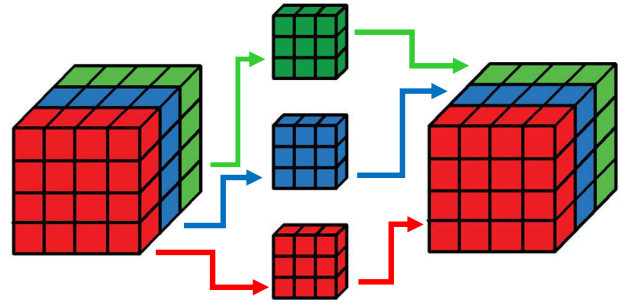


Fig.4 Depthwise convolution

The expression formula for Depthwise convolution is mentioned below

$$\text{Output}[i, j, k] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} \text{Input}[i+m, j+n, k] \times \text{Kernel}[m, n, k] \quad (2)$$

A pointwise convolution layer, often called a 1x1 convolution, is a specialized component in neural networks. It operates at the level of individual elements within feature

channels, distinct from standard convolutions that consider spatial relationships. This layer primarily focuses on transforming input channels independently using a 1×1 kernel size. It is valuable for dimensionality reduction or expansion, effectively altering the number of feature channels in the data. Pointwise convolutions are computationally efficient, reducing model complexity and computation, and nonlinear activation functions typically follow them. Moreover, they offer flexibility in feature combinations, allowing networks to learn intricate feature interactions.

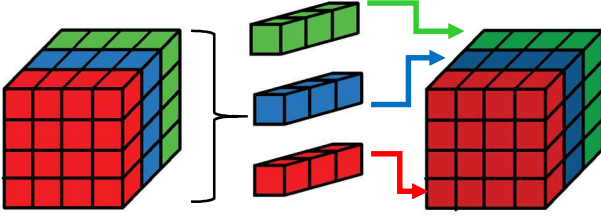


Fig.5 Pointwise convolution

The expression formula for Pointwise convolution is mentioned below.

$$\text{Output}[i, j, K] = \sum_{l=0}^{K-1} \text{Input}[i, j, l] \times \text{Kernel}[1, 1, l, K] \quad (3)$$

C. Batch Normalization

Batch normalization (BN) serves three primary purposes: expediting the training process and the convergence of convolutional neural networks, acting as a safeguard against overfitting, and managing the risk of gradient explosion. It achieves this by standardizing feature values across all samples, aligning them with a mean of zero and a variance of one. This adjustment centers the convolutional values within the effective range of nonlinear functions, preventing the issue of vanishing gradients. Following network training, the batch normalization layer can be described using Equation (5), with ' γ ' and ' β ' denoting constant values.

The Equation for normalizing input (x_i) is mentioned below,

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\alpha^2 + \epsilon}} \quad (4)$$

$\hat{x}_i$ is the normalized input using mean(μ) and variance(α)

$$\hat{y}_i = \gamma \hat{x}_i + \beta \quad (5)$$

Where γ and β are constants and $\hat{y}_i$ is the final output activation

D. ReLu Activation Function

ReLU (Rectified Linear Unit) activation, a cornerstone of deep neural networks, introduces non-linearity by zeroing out negative input values while preserving positive ones. This efficient function mitigates vanishing gradient issues and plays a pivotal role in the success of deep learning across diverse domains. Its simplicity and effectiveness in capturing complex patterns make it a fundamental choice in neural network architectures, as shown in Fig. 6.

The Equation for ReLu activation is mentioned below.

$$\text{ReLU}(x) = \min(6, \max(0, x)) \quad (6)$$

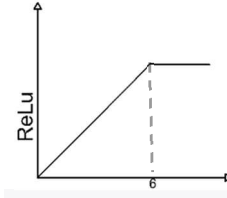


Fig.6 ReLu Activation

III. ALGORITHMS AND MODIFICATION

A. Depthwise Convolution

Algorithm 1 is a core component of deep learning known as Depthwise Convolution, commonly used in Convolutional Neural Networks (CNNs) for feature extraction. It operates on an input matrix ($W \times H$) and a kernel ($k \times k$) to yield an output matrix $(W-k+1) \times (H-k+1)$. Through systematic traversal of the output space, it computes convolutions via element-wise multiplication and accumulation. Depthwise convolutions are valued for their computational efficiency, often as initial CNN layers, followed by pointwise convolutions for intricate feature map creation. This algorithm is pivotal in advanced applications such as image processing and computer vision.

Algorithm 1 Depthwise Convolution

Required: Input [$W \times H$], Kernel [$k \times k$]

Expected: Output [$w \times h$], where $w = W - k + 1$ and $h = H - k + 1$

Initialize Output=0;

for $i=0$ to w do

for $j=0$ to h do

for $x=0$ to k do

for $y=0$ to k do

Output [i] [j] += Input [$i+x$] [$j+y$] $\times$ Kernel [x] [y]

end for

end for

end for

end

B. Batch Normalization

Algorithm 2 combines Batch Normalization and ReLU activation, enhancing deep neural network training and introducing non-linearity. It initializes an output tensor to zeros and standardizes input values per channel with Batch Normalization. This involves subtracting the channel mean, dividing by the channel variance plus a small epsilon, and scaling with gamma and beta. After rising, it applies the ReLU activation function, setting values below 0 to 0 and capping values above 6. This combination stabilizes training, learns channel-specific scaling and shifting, and introduces non-linearity, making it a common practice for improving deep learning model performance and stability.

Algorithm 2 Batch Normalization and Relu Function

Required: Input [$D \times W \times H$], beta [D], gamma [D], mean [D], variance [D]

Expected: Output [$D \times W \times H$]

Initialize Output=0;

for $ch=0$ to D do

for $x=0$ to W do

```

    for y=0 to H do
        normalized_value=(Input[ch][x][y]-
        mean[ch])/sqrt(variance[ch]+epsilon)
        scaled_value= gamma[ch]*
        normalized_value +beta[ch]
        if(scaled_value<0)
            Output[ch][x][y]=0
        else if(scaled_value<6)
            Output[ch][x][y]=6.0
        else
            Output[ch][x][y]= scaled_value;
        endif
    end for
end for
end for

```

C. Pointwise convolution

Algorithm 3 represents a Pointwise Convolution operation, a fundamental technique in deep learning extensively utilized within Convolutional Neural Networks (CNNs) to adapt and enhance feature maps. It operates on an input tensor of dimensions $D1 \times W \times H$ and a $D2 \times D1 \times 1 \times 1$ kernel, yielding an output tensor of dimensions $D2 \times W \times H$. The algorithm systematically traverses the width and height of the input feature maps, executing element-wise multiplications between the input tensor and kernel while accumulating results within the output tensor. This procedure is performed independently for each channel in the output tensor, serving as a pivotal transformation step to refine feature maps for subsequent neural network layers.

Algorithm 3 Pointwise Convolution

Required: Input $[D1 \times W \times H]$, Kernel $[D2] [D1]$
 Expected: Output $[D2 \times W \times H]$

Initialize Output=0;

```

for i=0 to W do
    for j=0 to H do
        for x=0 to D2 do
            for y=0 to D1 do
                Output[x][i][j] += Input[y][i][j] * Kernel[x*y+y]
            end for
        end for
    end for
end for
end for

```

IV. EXPLORATION AND OPTIMIZATION

Our methodology for efficient FPGA acceleration of MobileNet-V1 on the Zynq UltraScale MPSoC ZCU104 Evaluation Kit revolves around a strategic patch-based implementation. Both depthwise and pointwise convolutions are implemented in two stages, as shown in Fig. 7 and 8, with intermediate batch normalization and activation layers. Stage-1, shown in Fig. 7, utilizes a patch size of $9 \times 9 \times d$ from input feature map $M \times N$ stored in off-chip memory, which results in a 7×7 output dimension for depthwise convolution with depth 'd' (32 in our case). Here, a parallel structure is maintained to get d output patches parallel. These patches are sent to the batch normalization layer and relu blocks. Gamma,

beta, mean, and variance are stored in registers for quick access.

Stage 2, shown in Fig. 8, represents the pointwise convolution. Here, d (1024 as per the last layer in mobilenet-v1 model) sets of $d \times 1 \times 1$ weights are required. Only d1 sets (64 in our case) of $d \times 1 \times 1$ are read into on-chip memory and perform pointwise convolution, batch normalization and relu operation for effective utilization of on-chip memory. This stage is repeated until it generates all output feature maps.

The proposed architecture addresses the critical challenge of limited on-chip BRAM resources. We systematically dissect the MobileNet V1 architecture layer by layer, creating patches that encapsulate depthwise and pointwise convolution operations, batch normalization, and ReLU activation functions, treating them as a unified layer. A dynamic patch-switching mechanism ensures seamless transitions between patches during inference, preserving model accuracy. Effective resource management maximizes BRAM utilization while minimizing off-chip memory transfers. Each patch, loaded into BRAM, undergoes forward pass computations, delivering the desired output. Comprehensive power profiling validates the substantial energy savings achieved through our approach. Performance evaluation, including inference speed and model accuracy assessment, is benchmarked against non-patch-based implementations. Real-world experiments on the ZCU104 FPGA board validate the methodology's practical efficacy, and a comparative analysis showcases its superior power optimization. In conclusion, our meticulously designed patch-based method successfully addresses BRAM constraints. It significantly enhances power efficiency, making it an ideal choice for edge computing applications with stringent resource and power limitations.

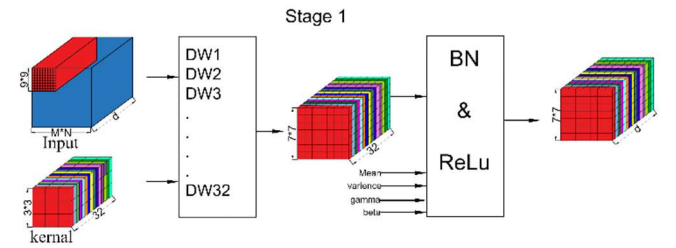


Fig.7 Patch-Based Depth Wise Convolution (Stage1)

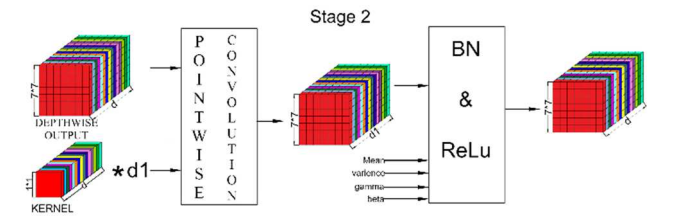


Fig.8 Patch Base Point Convolution (Stage 2)

The no of patches is given by $(n \times n)$ from the input feature map of size $M \times N$. Here, the expression for n is given by Eq.7.

$$n = \left(\frac{M-m}{m-k+1} \right) + 1 \quad (7)$$

$M \times N$ represents the size of the input feature map, m is the patch size (9 in our case), k is kernel size 3 and $n \times n$ is the

total number of slides to be made by the kernel on input feature map $M \times N$.

In above Table 2, the number of DSPs utilized in the proposed architecture is 520 $((32 \times 10) + 38 + 124 + 38)$. The 32 depthwise blocks shown in Fig. (7) are parallelized using 320 DSPs. The

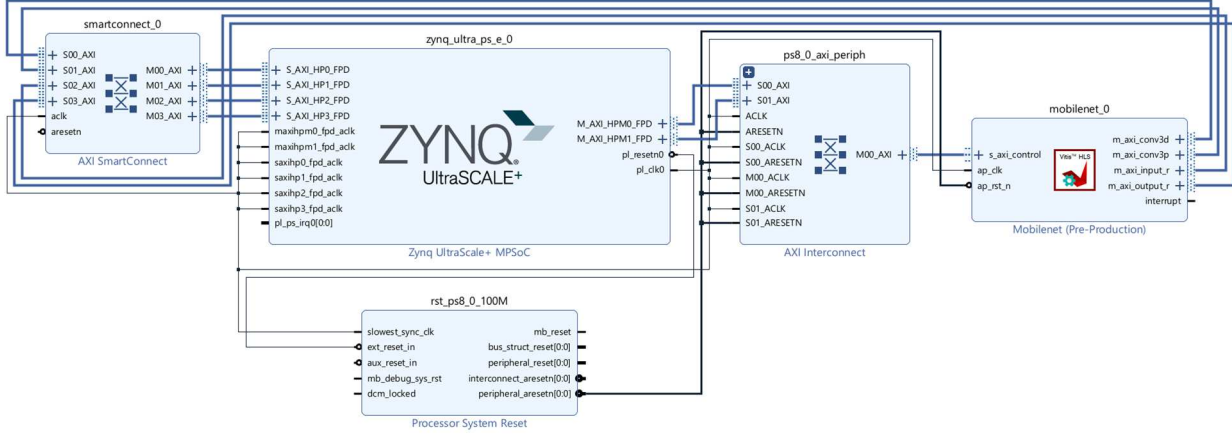


Fig.9 Block Design

Fig. 9 depicts the block design from the Vivado tool 2022.1 version. It integrates the proposed IP with the Zynq processor and AXI bus. The hardware setup is shown in Fig. 10.

V. RESULTS

In our experiments with the ZCU104 platform operating at 100 MHz, our optimized MobileNet implementation on the SAR dataset achieved impressive results (accuracy of 96.5%). We observed real-time inference performance, demonstrating its suitability for latency-critical applications, as shown in Fig. 10. Our patch-based optimization approach significantly improved Block RAM (BRAM) utilization while minimizing power consumption, making it an energy-efficient solution for embedded systems. These results showcase the potential for FPGA-based MobileNet deployments in resource-constrained environments, emphasizing the practicality of our approach for real-time object recognition.

Table 2: Resource Utilization

LAYERS	BRAM	DSP	LUT	FF
Conv DS-1	84	520	128747	128425
Conv DS-2	124	521	139975	128843
Conv DS-3	177	521	152563	131654
Conv DS-4	184	519	154273	131804
Conv DS-5	270	519	157896	132620
Conv DS-6	288	518	168404	138604
Conv DS-7	312	518	242327	145109
Conv DS-8	312	518	242327	145109
Conv DS-9	312	518	242327	145109
Conv DS-10	312	518	242327	145109
Conv DS-11	312	518	242327	145109
Conv DS-12	420	517	248579	154620
Conv DS-13	480	517	269420	197925

pointwise convolution used 124 DSPs, and the batch normalization layers used the remaining DSPs after the depthwise and pointwise convolution.

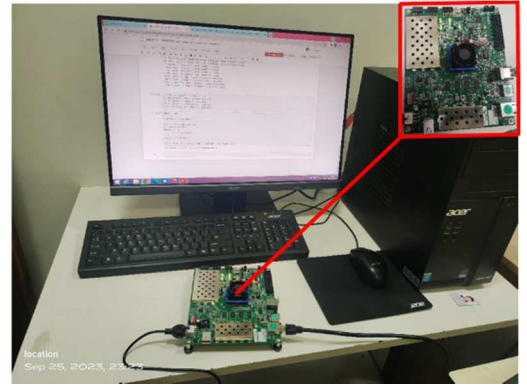


Fig.10 Hardware Implementation

Table 3: Number of Operations for Conv DS Layer 1

LAYERS	Number of Operations
Conv DW	218365952
Batch Normalization1	2408448
Conv PW	50577408
Batch Normalization1	4816896
Total Operations	276168704
Latency	3941156
Estimation	2.92
Throughput	24

Table 3 describes the number of operations that are more for depthwise and pointwise convolution when compared to batch normalization. The proposed architecture obtained 24 GOP/s by considering the square root and division (in batch normalization layer) as a single operation. Table 4 depicts the comparison of power with state-of-the-art works. The proposed architecture consumed less power, 5.0W.

Table 4: Comparison of Power with Other Implementations Based on MobileNet

Reference	GPU	[13]	[11]	[12]	This Paper
Model	MobileNet	MobileNet	MobileNet	MobileNet	MobileNet
Platform	Tesla K80 GPU	XCZU19EG	Xilinx Virtex-7 XC7V690t	ZU9EG	ZCU104
Power(W)	27.0	7.35	11.3	24	5.0

VI. CONCLUSION

This paper presented an FPGA-based optimization of the MobileNet architecture on the Zynq UltraScale+ MPSoC ZCU104 Evaluation Kit, focusing on a clock frequency of 100 MHz. The proposed patchwise design balanced resource utilization and power efficiency while maintaining high recognition accuracy. The experimental results highlight the viability of FPGA-based solutions for real-time SAR image classification. The effectiveness of the proposed architecture is further improved by implementing a fixed point type in future work. This work is a valuable reference for researchers and engineers seeking to deploy efficient computer vision solutions on FPGA platforms.

REFERENCES

- Howard, Andrew G., et al. "Mobilenets: Efficient convolutional neural networks for mobile vision applications." *arXiv preprint arXiv:1704.04861* (2017).
- Zhang, Xiao, and Jun Yang. "A High-Performance FPGA-based Accelerator for MobileNet." *2022 International Conference on Informatics, Networking and Computing (ICINC)*. IEEE, 2022.
- Bouguezzi, S., et al. "An Efficient FPGA-Based Convolutional Neural Network for Classification: Ad-MobileNet. Electronics 2021, 10, 2272." (2021).
- Wu, Di, et al. "A high-performance CNN processor based on FPGA for MobileNets." *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2019.
- Chen, Hong-Yen, and Chung-Yen Su. "An enhanced hybrid MobileNet." *2018 9th International Conference on Awareness Science and Technology (iCAST)*. IEEE, 2018.
- Ryu, Sungju, Youngtaek Oh, and Jae-Joon Kim. "MobileWare: A high-performance mobilenet accelerator with channel stationary dataflow." *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2021.
- Xiao, Jiale, Yonghao Chen, and Tao Su. "A mobilenet accelerator with high processing-element-efficiency on FPGA." *2021 China Semiconductor Technology International Conference (CSTIC)*. IEEE, 2021.
- Su, Jiang, et al. "Redundancy-reduced mobilenet acceleration on reconfigurable logic for imagenet classification." *Applied Reconfigurable Computing. Architectures, Tools, and Applications: 14th International Symposium, ARC 2018, Santorini, Greece, May 2-4, 2018, Proceedings 14*. Springer International Publishing, 2018.
- Bai, Lin, Yiming Zhao, and Xinming Huang. "A CNN accelerator on FPGA using depthwise separable convolution." *IEEE Transactions on Circuits and Systems II: Express Briefs* 65.10 (2018): 1415-1419.
- Yanamala, R. M. R., & Pullakandam, M. (2022). An Efficient Configurable Hardware Accelerator Design for CNN on Low Memory 32-Bit Edge Device. In 2022 IEEE International Symposium on Smart Electronic Systems (iSES) (pp. 1-5). IEEE.
- Yan, Shun, et al. "An FPGA-based mobilenet accelerator considering network structure characteristics." *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2021.
- Sada, Youki, et al. "A dataflow pipelining architecture for tile segmentation with a sparse MobileNet on an FPGA." *2019 International Conference on Field-Programmable Technology (ICFPT)*. IEEE, 2019.
- Pérez, Ignacio, and Miguel Figueroa. "A heterogeneous hardware accelerator for image classification in embedded systems." *Sensors* 21.8 (2021): 2637.